

PROTOCOL SPECIFICATION

Challenge / Review Protocol Layer v0.1

Procedural layer for dispute, challengeability, review, settlement, and bounded reinterpretation in the Triad Stack

STATUS

Concept Draft

POSITION IN STACK

Follows `Witness Overlay / Evidence Notation Layer`; governs how witnessed objects can be contested without collapsing continuity, auditability, or L4 discipline

Kotov Ivan

Bruxelles 2026

Prepared from the source
Markdown with author attribution added for publication packaging.

1. Purpose

A graph that shows evidence but does not define how evidence may be contested remains incomplete.

A witnessed object may still be:

- misclassified,
- context-incomplete,
- prematurely settled,
- bound to stale assumptions,
- or visually persuasive while operationally unsafe.

The system therefore requires a dedicated **Challenge / Review Protocol Layer**.

Its role is not to erase witness. Its role is not to allow aesthetic override. Its role is not to permit endless argument.

Its role is to formalize:

- who may challenge,
 - what may be challenged,
 - under what time horizon,
 - how review proceeds,
 - how outcomes are recorded,
 - and how settlement occurs without destroying historical trace.
-

2. Core Principle

Witness makes an event visible.

Challenge makes interpretation accountable.

A long-lived system must therefore distinguish:

- event occurrence,

- event classification,
- event legitimacy,
- and event consequence.

Those are not identical.

A system that cannot be challenged becomes dogmatic. A system that can be challenged without limits becomes unstable.

The Challenge / Review Protocol Layer exists to preserve the narrow path between those failures.

3. What May Be Challenged

The following objects are challengeable by default unless explicitly marked otherwise.

3.1 Node Classification

Examples:

- was this really an `EnergyLock` ?
- should this node be `ResearchNode` rather than `DeadEndNode` ?
- is this branch incorrectly marked `cinematic_only` ?

3.2 Edge Meaning

Examples:

- does this transition really imply irreversible failure?
- was this fork generated from the correct prior node?

3.3 Evidence Interpretation

Examples:

- is the witness sufficient?
- is the evidence state incorrectly elevated from `observed` to `witnessed` ?

- does the signature prove integrity but not ontological correctness?

3.4 Scope of Legitimacy

Examples:

- does this evidence support audit only, or runtime reliance?
- is this witness stale relative to current privilege context?

3.5 Branch Eligibility for Reopening

Examples:

- has enough new evidence arrived?
 - may a quarantined node move to `reopenable` ?
-

4. What May NOT Be Challenged

Certain realities remain non-negotiable.

4.1 L4 Physical Outcomes

Examples:

- thermal shutdown already occurred,
- time window has closed,
- hardware failed,
- privilege boundary blocked execution.

Interpretation around these events may be challenged. The physical event itself may not be rhetorically reversed.

4.2 Historical Witness Existence

A witness event that occurred may not be erased through review.

It may be:

- reclassified,
- contextualized,

- marked disputed,
- marked superseded,
- marked expired.

But not silently deleted.

4.3 Authority Through Interface Pressure

No amount of review UI activity may grant execution power to an object that remains quarantined.

5. Challenge Roles

The protocol must define who is allowed to challenge.

5.1 `human_anchor`

The accountable `a` associated with `c`.

Rights:

- open challenge,
- inspect evidence,
- request review,
- veto settlement if policy grants it.

5.2 `entity_internal_reviewer`

A bounded review role inside the long-lived system.

Rights:

- re-evaluate evidence classes,
- compare branch histories,
- recommend status changes,
- may not self-authorize execution.

5.3 `external_reviewer`

A privileged outside reviewer, auditor, or expert steward.

Rights:

- review contested evidence,
- attach review outcome,
- issue signed challenge decisions if authorized.

5.4 `macro_steward`

System-level or institutional steward.

Rights:

- review policy meaning,
- interpret challenge scope,
- define procedural constraints,
- not exempt from witness logging.

5.5 `interface_observer`

A read-only viewer.

Rights:

- inspect,
 - annotate privately,
 - no challenge initiation unless upgraded.
-

6. Challenge Lifecycle

Every challenge moves through a bounded lifecycle.

6.1 `open`

Challenge is filed.

Requirements:

- explicit challenge target,
- challenger role,
- challenge reason,
- timestamp,

- scope tag.

6.2 **queued**

Challenge accepted into the review pipeline but not yet evaluated.

6.3 **under_review**

Evidence and classification actively assessed.

6.4 **resolved_uphold**

Original interpretation upheld.

6.5 **resolved_modify**

Interpretation changed.

Examples:

- node reclassified,
- evidence status downgraded,
- challenge scope narrowed,
- expiration attached.

6.6 **resolved_split**

Challenge reveals that one prior object should become two.

Example:

- one node remains historically witnessed,
- a second node reflects the revised research path.

This is especially important to avoid destructive rewriting.

6.7 **dismissed**

Challenge rejected for procedural insufficiency.

6.8 **expired**

Challenge window elapsed before resolution.

6.9 `archived`

Challenge process completed and retained as historical trace.

7. Challenge Windows

Challengeability must be time-bound.

Without windows, the graph never stabilizes. Without challengeability, the graph calcifies too early.

7.1 Challenge Window Fields

Every challengeable object may expose:

- `challenge_opened_at`
- `challenge_deadline`
- `challenge_scope`
- `challenge_role_required`

7.2 Window Types

`short_window`

For low-level runtime classification.

`extended_window`

For research classification and evidence interpretation.

`institutional_window`

For audit or governance disputes.

7.3 Closure Rule

After challenge deadline:

- object may become `settled`,
 - but historical openness must remain visible in audit mode.
-

8. Review Outcomes

A review must never silently rewrite history.

Permitted outcome forms:

8.1 `annotation`

Adds interpretive context without changing node class.

8.2 `reclassification`

Changes node or edge category.

8.3 `evidence_downgrade`

Example:

- from `signed` to `witnessed`,
- from `witnessed` to `observed`.

8.4 `evidence_upgrade`

Only with new witness material.

8.5 `branch_split`

Creates two visible continuations:

- original historical branch,
- revised interpretation branch.

8.6 `reopenability_change`

Changes whether a quarantined object may move toward `reopenable`.

8.7 `scope_restriction`

Example:

- audit-use only,
- historical reference only,
- no runtime reliance.

9. Red Lines

The protocol must explicitly block the following anti-patterns.

9.1 Aesthetic Override

A visually persuasive walkthrough may not settle a challenge.

9.2 Silent Rewrite

Resolved review may not overwrite original branch memory without preserved trace.

9.3 Authority Laundering Through Review

Winning a review does not automatically create action authority.

9.4 Endless Review Loops

Challenges must be finite.

Repeated reopening requires new evidence or new context.

9.5 Collapse of Evidence Into Opinion

Commentary, intuition, and cinematic plausibility may support review discussion.

They may not replace witness material.

10. Challenge Objects

The layer should define a distinct challenge object.

Minimal schema:

```
{
  "challenge_id": "chg_01H...",
  "target_ref": "node_01H...",
  "challenge_type": "classification_dispute",
  "opened_by": "human_anchor",
  "reason": "insufficient evidence for signed status",
  "opened_at": "2026-04-01T18:00:00Z",
  "deadline": "2026-04-08T18:00:00Z",
  "status": "under_review",
  "review_outcome": null,
  "review_ref": null,
  "new_evidence_refs": []
}
```

Challenge objects are first-class graph attachments, not hidden log events.

11. Review Objects

Each review should also be explicit.

Minimal schema:

```
{
  "review_id": "rev_01H...",
  "challenge_ref": "chg_01H...",
  "reviewer_role": "external_reviewer",
  "outcome": "resolved_modify",
  "action": "reclassify_node",
  "previous_class": "DeadEndNode",
  "new_class": "ResearchNode",
  "signed": true,
  "witness_ref": "wit_01H...",
  "resolved_at": "2026-04-04T12:00:00Z"
}
```

12. Visual Conventions for Challenge / Review

The graph should make challenge status legible.

12.1 Challenge Markers

- open challenge → amber ring / warning contour
- under review → amber dashed halo
- resolved_uphold → green check overlay
- resolved_modify → blue branch-split marker or blue revision tag
- dismissed → gray strike marker
- expired challenge → faded amber slash

12.2 Review Trails

Audit mode should show:

- who challenged,
- who reviewed,
- whether signed,
- whether branch split occurred.

12.3 Split Rendering

If review causes branch split:

- historical branch remains visible,
- revised branch emerges adjacent,
- relation marked as `review_split`.

This avoids the false impression that the original path never existed.

13. Relation to Witness Overlay

Witness Overlay answers:

- what evidentiary status is visible?

Challenge / Review answers:

- how may that status be contested and revised?

The two layers must remain separate.

A node may be:

- strongly witnessed,
- but still challenge-open,
- then later settled,
- then later expired.

This progression must remain visible.

14. Relation to Triad Stack

14.1 Core

The Core may emit witnessable events.

It does not settle disputes by runtime assertion.

14.2 Research Layer

Research nodes are especially likely to be challengeable.

Examples:

- should this stay quarantined?
- is this truly reopenable?
- is the proposed bridge evidentially too weak?

14.3 Cinematic Layer

The cinematic layer may present a challenge path.

It may not decide the review outcome.

15. Why This Matters for Long-Lived Systems

Long-lived systems do not fail only because they hallucinate.

They also fail because they:

- freeze bad classifications,
- quietly rewrite prior uncertainty,
- confuse witnessed history with final truth,
- or allow review theater to replace disciplined reinterpretation.

The Challenge / Review Protocol Layer prevents that.

It ensures that continuity does not mean rigidity.

And that revision does not mean amnesia.

16. Explicit Bridge

This layer makes one principle procedural:

A witnessed world must remain challengeable, but not endlessly unstable.

That is the bridge between:

- witness,
 - research quarantine,
 - and continuity that survives disagreement.
-

17. Hidden Bridges

Hidden Bridge 1 — Cybernetics

A regulator must be corrigible without losing coherence.
Challenge / Review is the bounded corrigibility layer.

Hidden Bridge 2 — Information Theory

Revision without preserved trace destroys signal lineage.
Challenge / Review preserves interpretive mutation
without collapsing provenance.

18. Earth Paragraph

In engineering, a failed pressure reading may trigger an inspection, a second measurement, and a signed maintenance decision. What it does not permit is quietly repainting the gauge so the problem “looks solved.” A serious AI graph needs the same discipline: dispute, inspect, revise if needed, but never cosmetically erase the fact that something broke and had to be reviewed.

19. Final Position

The Challenge / Review Protocol Layer turns witness from static evidence into procedural discipline.

It defines:

- who may contest,
- what may be contested,
- how long challenge remains open,
- how reinterpretation happens,
- and how the graph remembers disagreement without collapsing into chaos.

Without this layer, witness remains visible.

But it does not yet become governance-capable con